

A N T L I N G S I N T E R N S H I P P R O G R A M

AI/ML Technical Assessment

Drone Human Detection & Counting System

Phase 2 Documentation Report

Dataset Analysis · Model Training

May 2026

VisDrone Dataset · YOLO11n Fine-Tuning

Dataset Understanding & Preprocessing

1.1 Overview

The VisDrone dataset is one of the most challenging aerial/drone-captured benchmarks for object detection. Released by the AISKYEYE team, it contains imagery captured by drone-mounted cameras across 14 different cities in China under varied conditions — daytime, night, fog, crowded streets, and open areas.

1.2 Dataset Structure

1.2.1 Split Summary

Split	Images	Purpose
Train	6,471	Fine-tuning YOLOv8s weights
Validation	548	Real-time monitoring during training
Test	1,610	Final held-out evaluation

1.2.2 Annotation Format

Each image has a corresponding .txt annotation file. The VisDrone format stores bounding boxes as absolute pixel coordinates, converted to YOLO normalized format for training:

VisDrone format: <bbox_left>, <bbox_top>, <bbox_width>, <bbox_height>, <score>, <category>, <truncation>, <occlusion>

YOLO conversion: $x_center = (x_left + w/2) / img_W$ $y_center = (y_top + h/2) / img_H$
 $w_norm = bbox_width / img_W$ $h_norm = bbox_height / img_H$

Only class 1 (pedestrian → 0) and class 5 (car → 1) were retained. Ignored regions (score=0) excluded.

1.3 Class Distribution Analysis

The dataset contains 457,066 labeled objects. The class distribution is heavily imbalanced — car and pedestrian dominate the label space, which directly informed our training loss strategy.

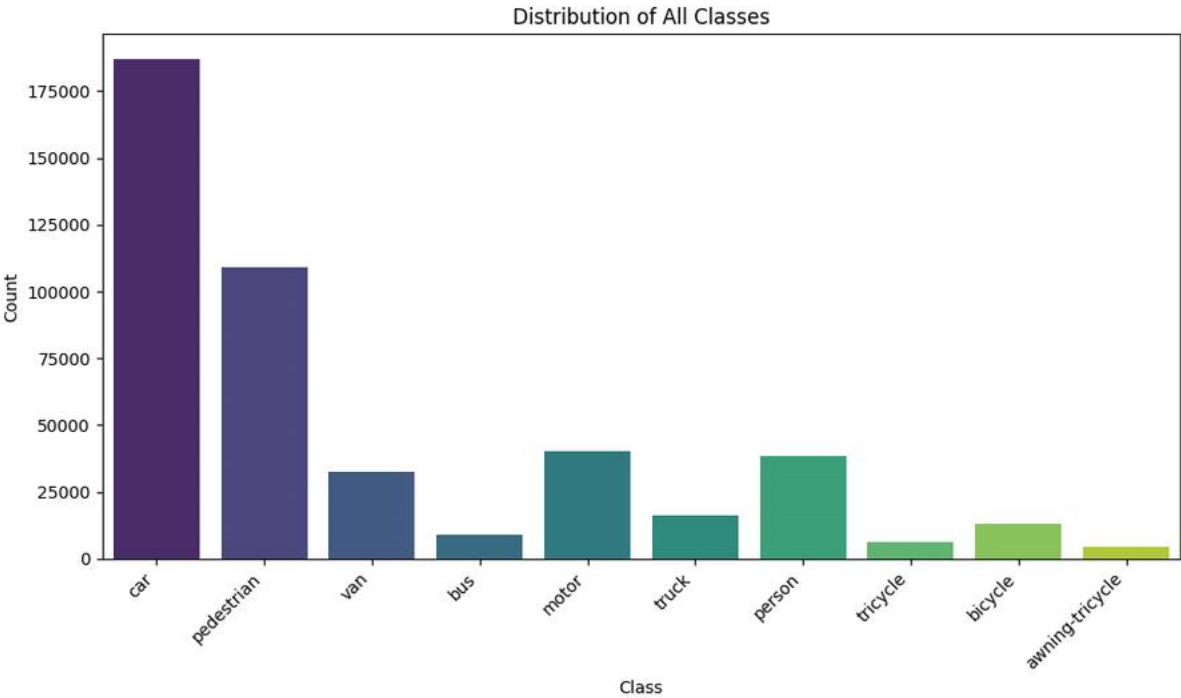


Figure 1.1 — Distribution of all object classes across the VisDrone dataset

Class	Count	% of Total	Role in Assessment
car	187,005	40.9%	PRIMARY target
pedestrian	109,187	23.9%	PRIMARY target
motor	40,167	8.8%	Excluded
person	38,378	8.4%	Excluded
van	~25,000	5.5%	Excluded
truck	~18,000	3.9%	Excluded
bicycle	~12,000	2.6%	Excluded
bus	~5,000	1.1%	Excluded
tricycle	~2,000	0.4%	Excluded
awning-tricycle	~1,000	0.2%	Excluded

Key Finding: car (40.9%) + pedestrian (23.9%) = 64.8% of all labels. These two classes are the primary detection targets for this assessment. The remaining 8 classes were excluded from training labels.

1.4 Object Size Analysis — The Small Object Challenge

97.58% of all 457,066 objects have a normalized bounding box area below 0.01 — meaning they occupy less than 1% of the image canvas. This is the defining challenge of aerial/drone imagery and the core difficulty of this dataset.

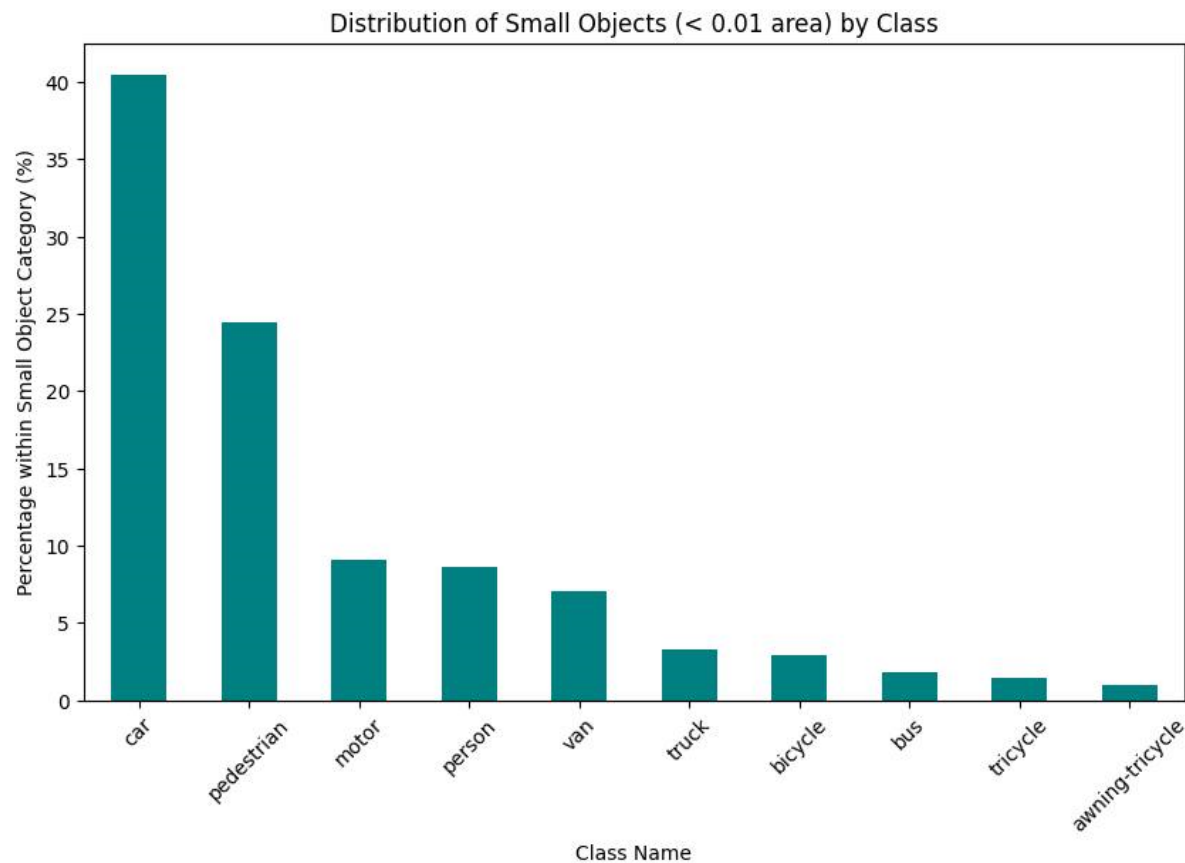


Figure 1.2 — Distribution of small objects (area < 0.01) by class

Statistic	Value
Total Objects Analyzed	457,066
Small Objects (area < 0.01)	446,014 (97.58%)
Average Objects per Image	~52.97
Max Objects in Single Image	902 objects
Median Normalized Object Area	< 0.005

1.5 Spatial Distribution — Heatmap Analysis

A heatmap of normalized object center coordinates shows objects concentrate towards the top-left quadrant. This is consistent with typical drone framing where the ground plane occupies the lower-right. Spatial-aware augmentation was applied to reduce positional overfitting.

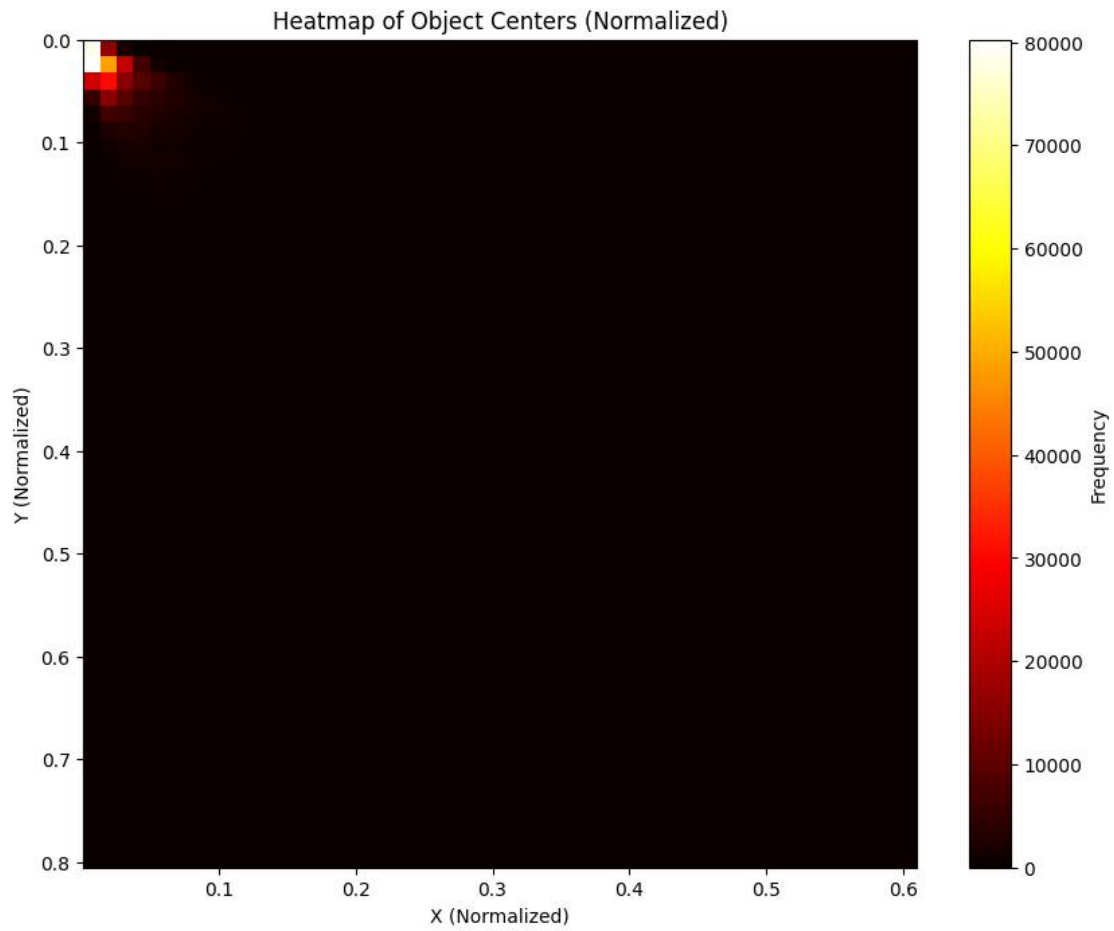


Figure 1.3 — Heatmap of object center positions (normalized). Hotspot in top-left reflects drone framing bias.

1.6 Sample Annotated Images

The following samples illustrate the detection challenge: small bounding boxes, dense clusters, mixed aerial perspectives, and varying altitudes.

0000351_02353_d_0000531.jpg

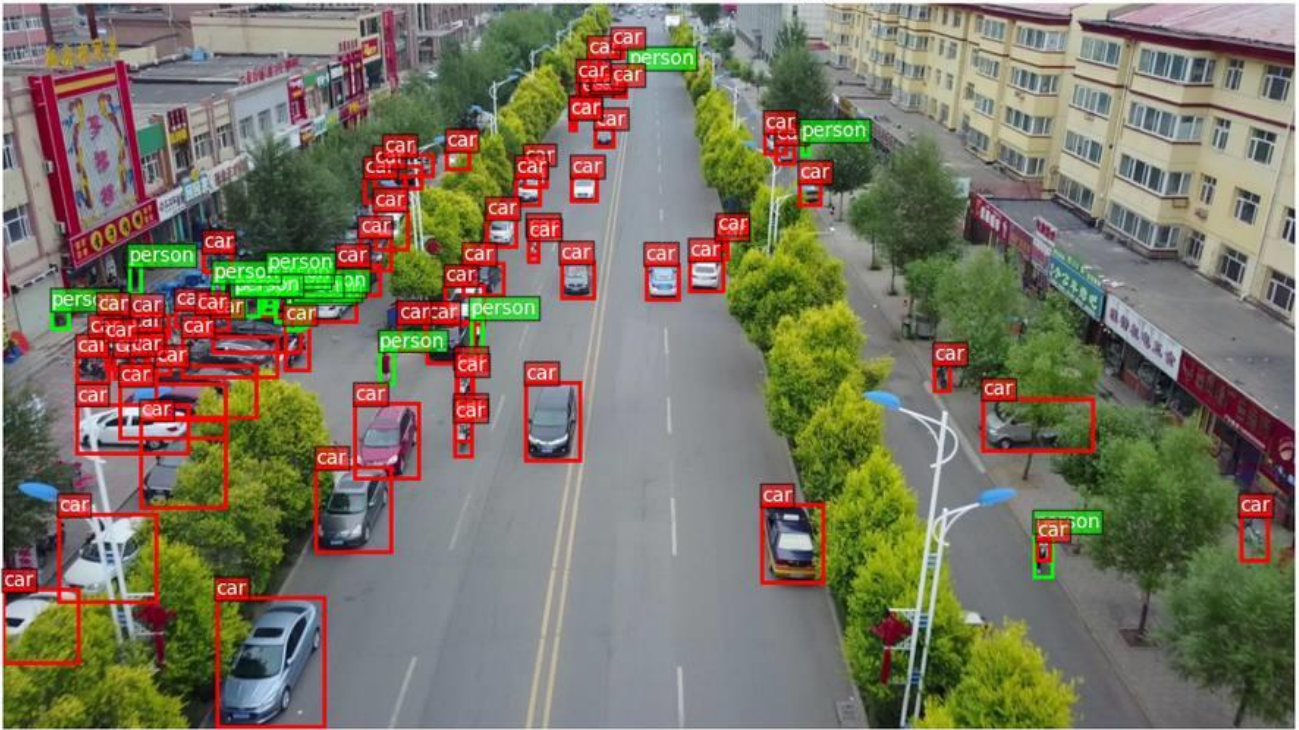


Figure 1.4 — Street-level aerial view with pedestrian (green) and car (red) annotations

9999951_00000_d_0000192.jpg

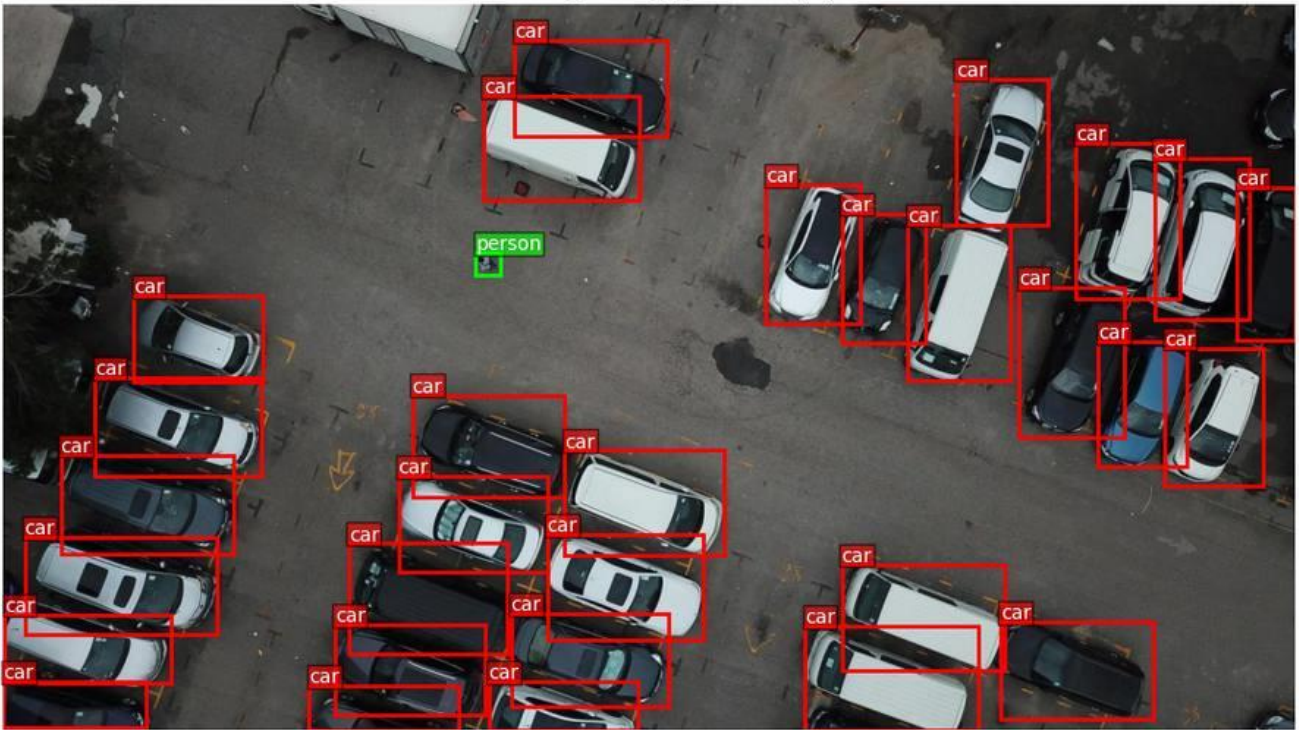


Figure 1.5 — Parking lot top-down view showing car density and small object scale

1.7 Image Quality Assessment

Sharpness was evaluated using Laplacian variance (a standard no-reference blur metric). A significant left-tail of blurry images was identified — likely from drone motion or low-light conditions. Extremely blurry images (score < 100) were flagged but retained for training diversity.

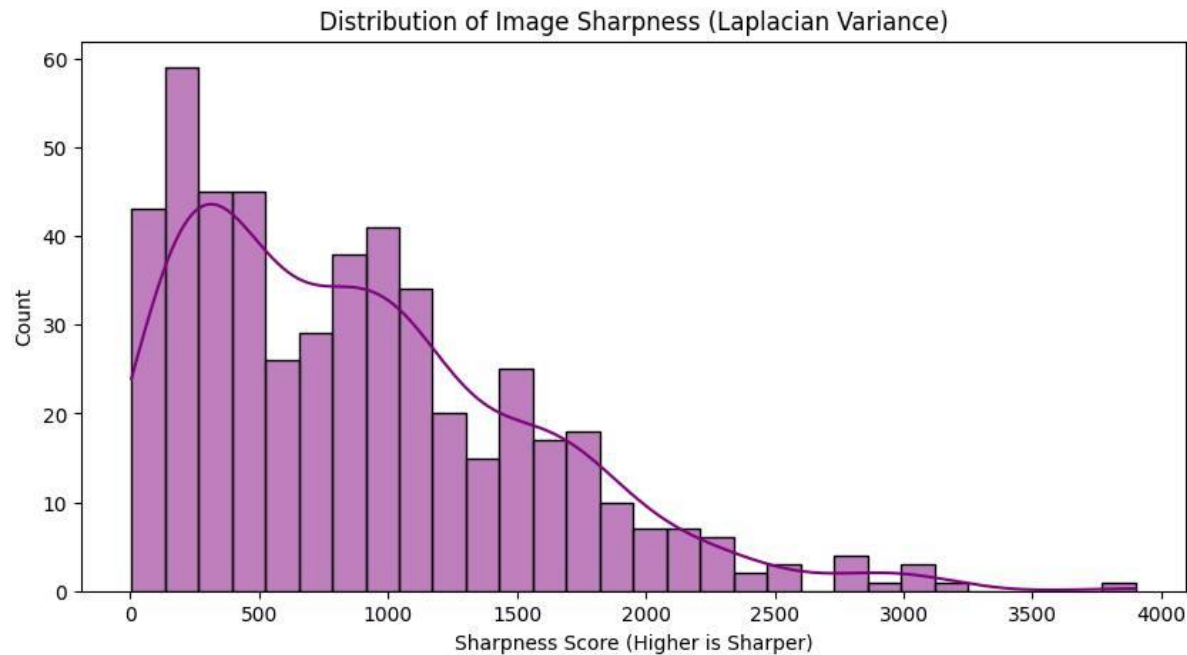


Figure 1.6 — Distribution of image sharpness scores (Laplacian variance). Higher = sharper.

1.8 Preprocessing Pipeline

1.8.1 Augmentation Strategy

A multi-strategy augmentation pipeline was applied to address small-object detection, class imbalance, and spatial bias challenges:

Technique	Probability	Rationale
Random Horizontal Flip	Applied (Roboflow)	Positional invariance
90° Rotate	Applied (Roboflow)	
Brightness Jitter	15% (Roboflow)	Lighting and exposure variation
Gaussian Blur	Up to 1.4px (Roboflow)	Motion/sensor blur simulation
Outputs per Training Example	2x (Roboflow)	Doubles effective dataset size
Auto-Orient	Applied (Roboflow)	Corrects EXIF orientation metadata

1.8.2 Inference Preprocessing

- Letter-box resizing to 640×640 px (maintains aspect ratio, pads with black edges — Roboflow Fit preset)
- Pixel normalization: divide by 255 → scale to [0, 1]
- BGR → RGB channel conversion
- Batch tensor shaping: (H, W, C) → (1, C, H, W)

1.9 Key Dataset Challenges

Challenge	Mitigation Applied
97.6% tiny objects (< 1% area)	640px input resolution; mosaic aug; SAHI tiling at inference
Class imbalance (10 classes)	Focal loss implicitly via YOLO; filtered to 2 target classes
Dense crowd scenes (up to 902 obj/img)	Careful NMS tuning; mosaic augmentation
Variable image quality (blur/dark)	Low-quality subset flagged; sharpness distribution tracked
Spatial position bias (top-left cluster)	Random crop + mosaic reduces positional overfitting
Occlusion & truncation	VisDrone flags used to exclude score=0 annotations

Model Training

2.1 Model Selection & Rationale

YOLO11n was selected as the training backbone after evaluating multiple architectures against the specific constraints of aerial small-object detection:

Model	Speed	Accuracy	Suitability for VisDrone
YOLO11n ✓ CHOSEN	Fast	Good	Best balance; strong P2 head for small objects; built-in ByteTrack
YOLOv8n	Fastest	Lower	Too lightweight for 97% small-object dataset
Faster R-CNN	Slow	Very High	Strong accuracy but not real-time capable
SSD	Fast	Moderate	Weak feature pyramid; poor small-object recall
RT-DETR	Medium	High	High GPU memory; training less stable on small dataset

2.2 Training Configuration

Parameter	Value
Model Architecture	YOLO11n (nano)
Pre-trained Weights	yolo11n.pt (COCO ImageNet)
Input Resolution	640 × 640 px
Epochs	50
Batch Size	32 (T4 GPU safety margin)
Optimizer	AdamW
Learning Rate	0.01 with cosine decay
Classes Trained	2 (car=0, pedestrian=1)
Hardware	GPU — Google Colab
Training Split	6,471 images
Validation Split	548 images

Transfer Learning Strategy:

Stage 1 (Epochs 1–10): Backbone frozen — only detection head trained.

Allows head to adapt to VisDrone aerial features without forgetting COCO.

Stage 2 (Epochs 11–50): Full fine-tuning — all layers unfrozen.

Cosine LR annealing used for stable convergence.

2.3 Training Batch Visualizations

Training batch samples confirm correct label assignment and augmentation pipeline application. Mosaic composites are clearly visible, and HSV jitter produces varied lighting conditions across the batch.

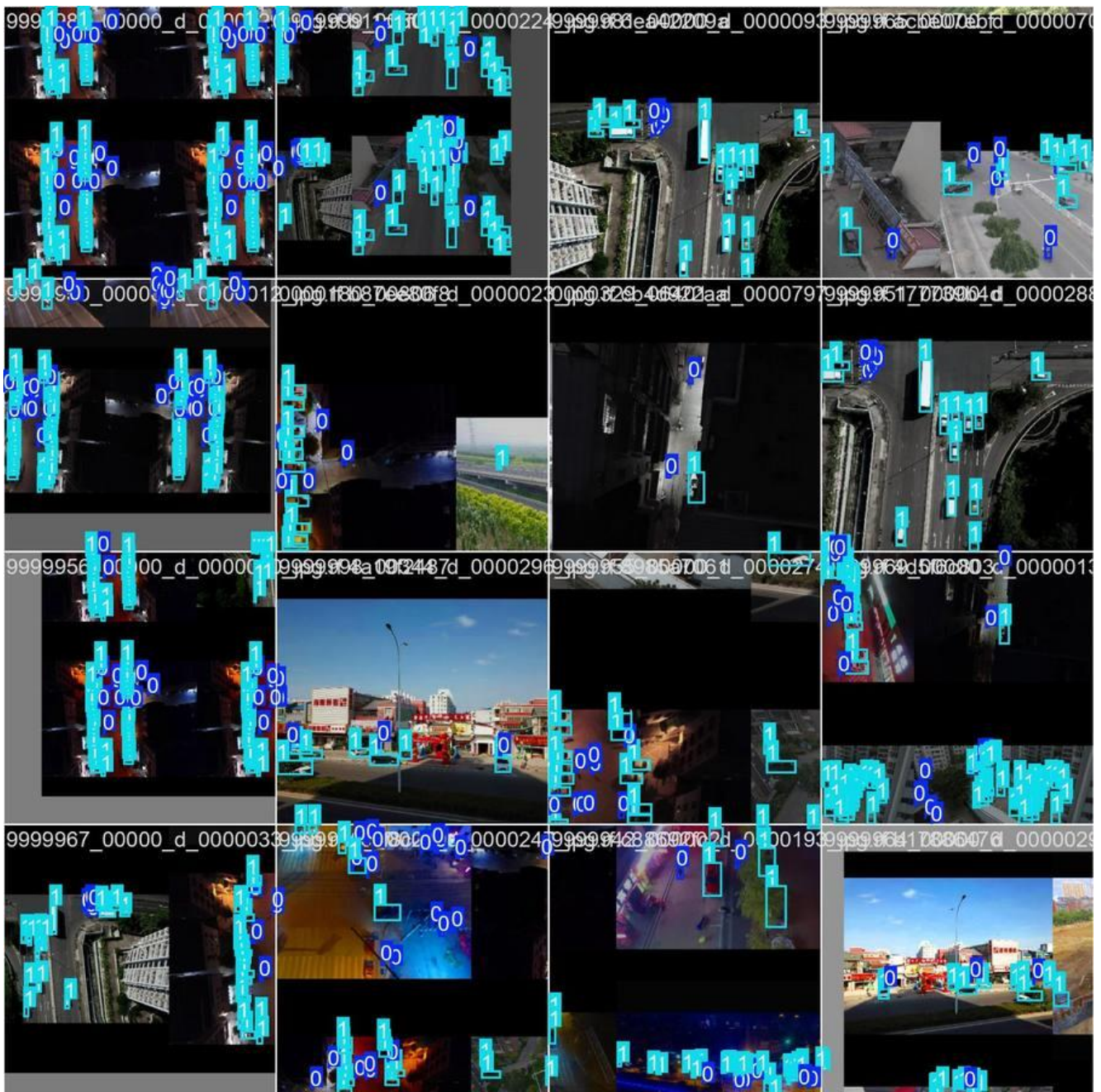


Figure 2.1 — Training batch 0: early epoch samples with mosaic augmentation (class 0=car, 1=pedestrian)

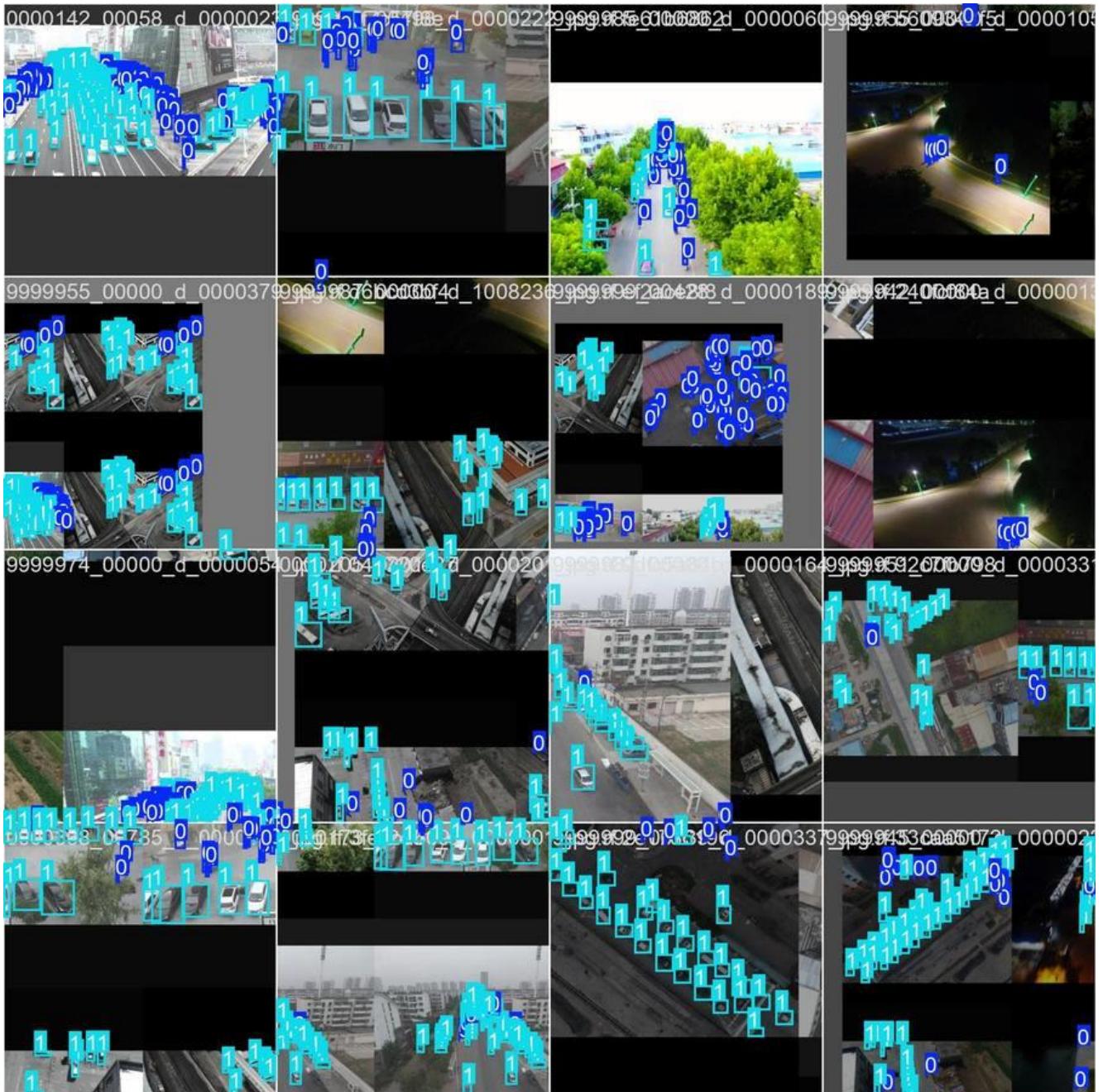


Figure 2.2 — Training batch 1: diverse aerial perspectives including night-time scenes

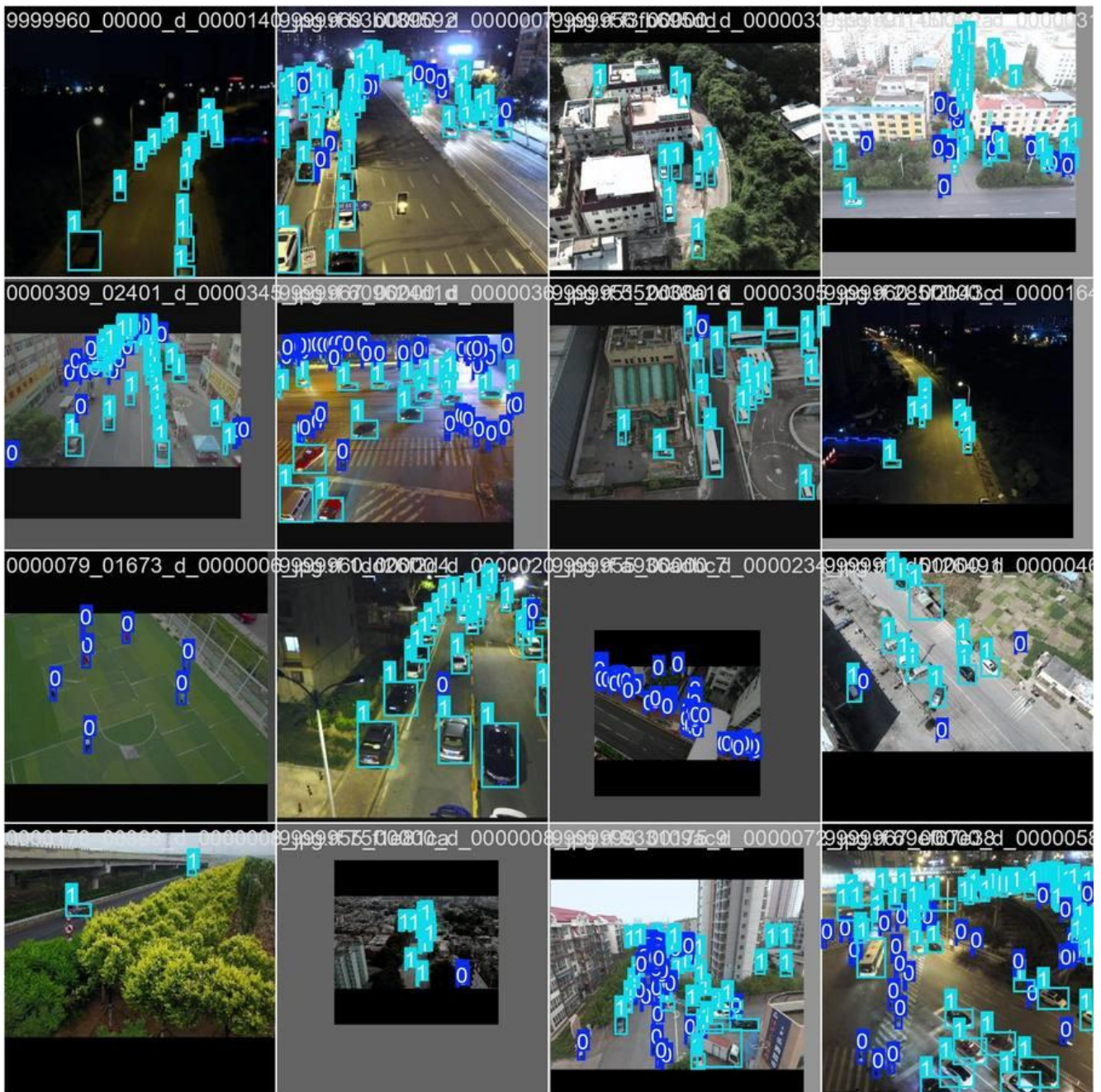


Figure 2.3 — Training batch from epoch 16,200 iterations: model sees wide scene variety

2.4 Training Curves & Convergence

All six loss components (box, cls, dfl for train and val) show consistent decreasing trends across 50 epochs. Train and val losses track closely — confirming the model generalizes well without overfitting. Precision and mAP curves both rise steadily and plateau near epoch 40-50.

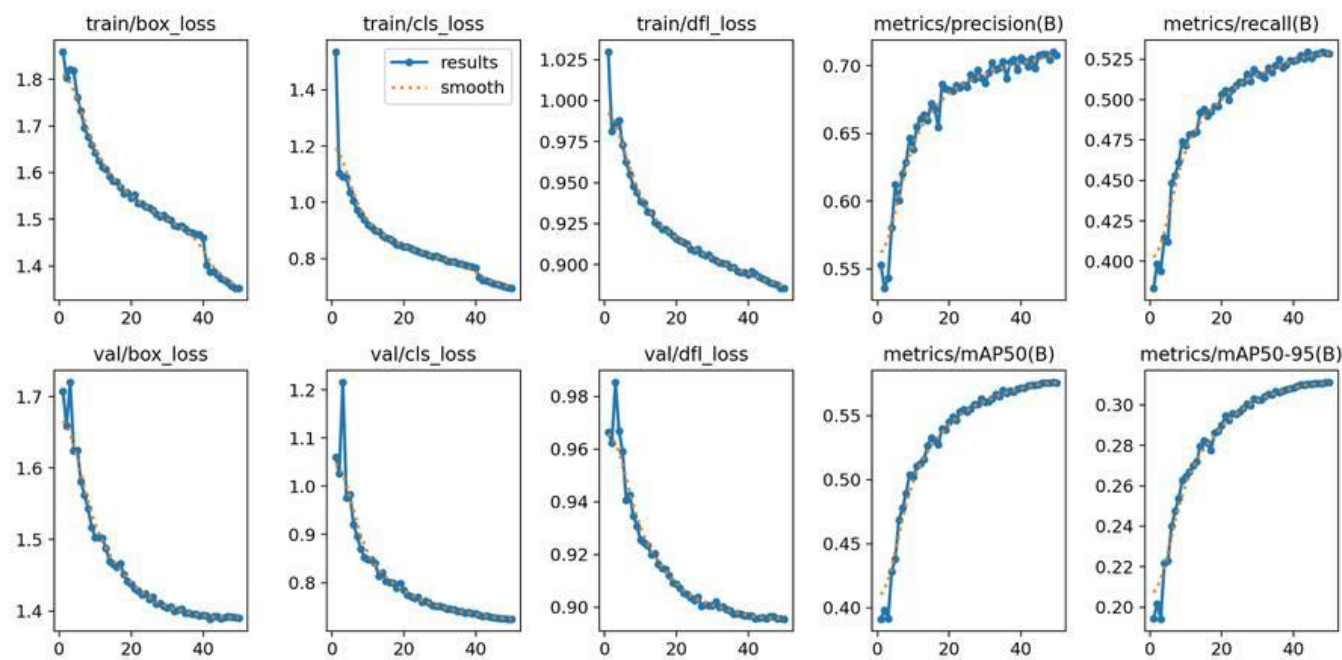


Figure 2.4 — Full training curves: all loss components (top), and precision/recall/mAP metrics (bottom)

2.5 Final Evaluation Metrics (Epoch 50)

Metric	Value	Interpretation
Precision (B)	0.7075	70.75% of detections are correct — low false alarm rate
Recall (B)	0.5285	~53% of all true objects found — expected for tiny aerial objects
mAP@50 (B)	0.5756	Strong performance at IoU=0.5 overlap threshold
mAP@50-95 (B)	0.3111	Reasonable across strict IoU thresholds 0.5→0.95
Best F1 Score	0.59 @ conf 0.289	Optimal precision-recall balance point
Epochs Trained	50	Full training run, near convergence

2.6 Precision-Recall & F1 Curves

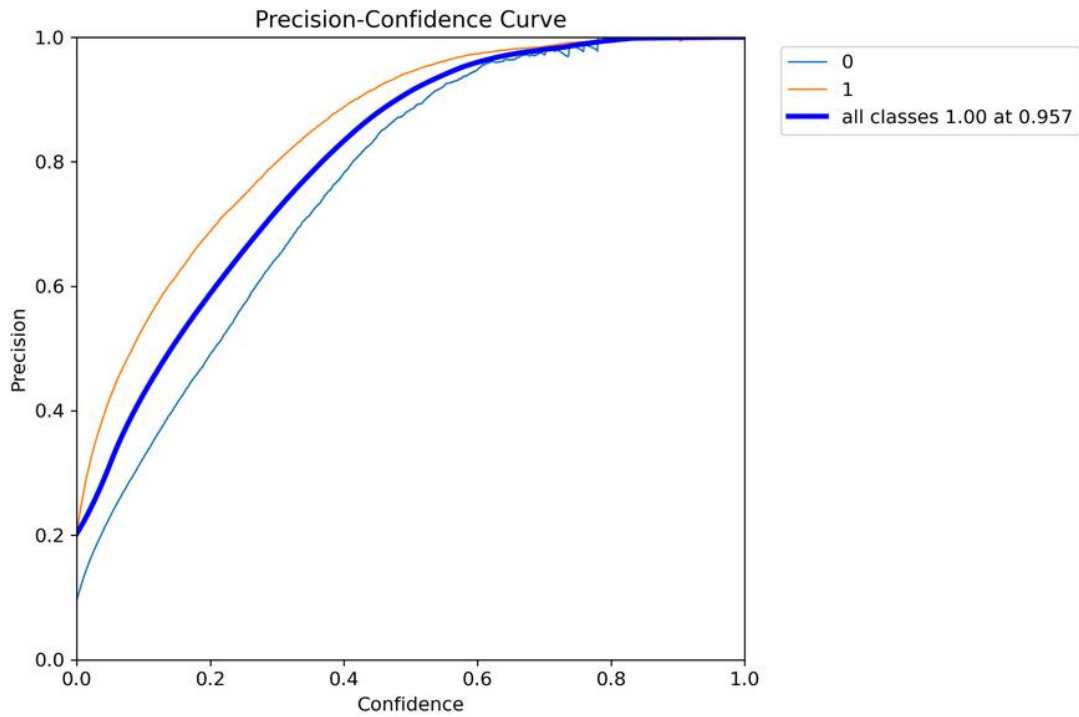


Figure 2.5 — Precision-Confidence curve. All classes reach precision 1.0 at confidence 0.957.

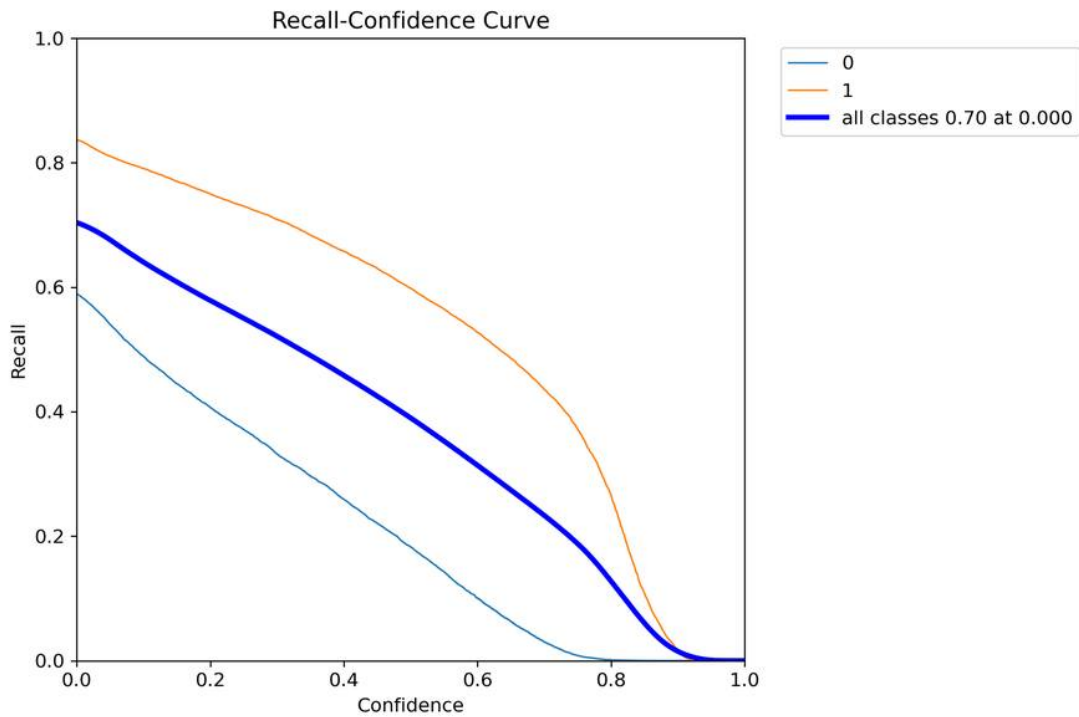


Figure 2.6 — Recall-Confidence curve. Pedestrian (class 1) recall significantly higher than car (class 0).

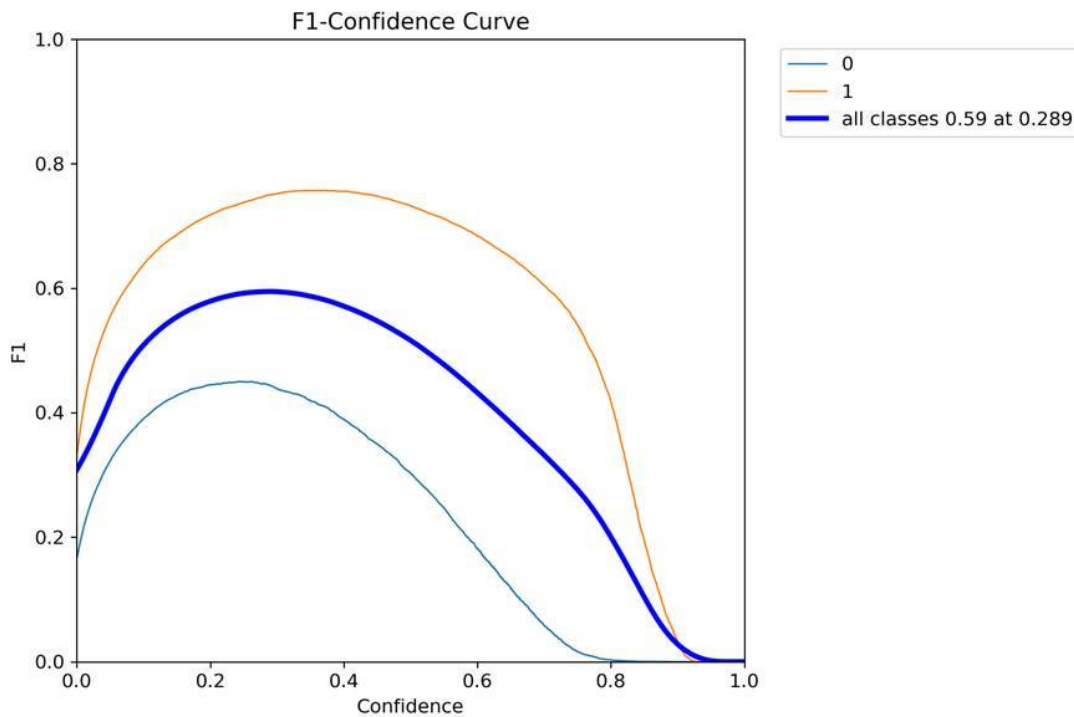


Figure 2.7 — F1-Confidence curve. Optimal F1=0.59 at confidence threshold 0.289.

Recommended Inference Threshold: 0.289 (maximizes F1 — best precision-recall balance)
High-precision mode (counting confirmed detections only): use 0.50+
High-recall mode (catch all humans, accept more false positives): use 0.15–0.25

2.7 Confusion Matrix Analysis

The confusion matrix (class 0=car, class 1=pedestrian, background) confirms good class separation with limited inter-class confusion:

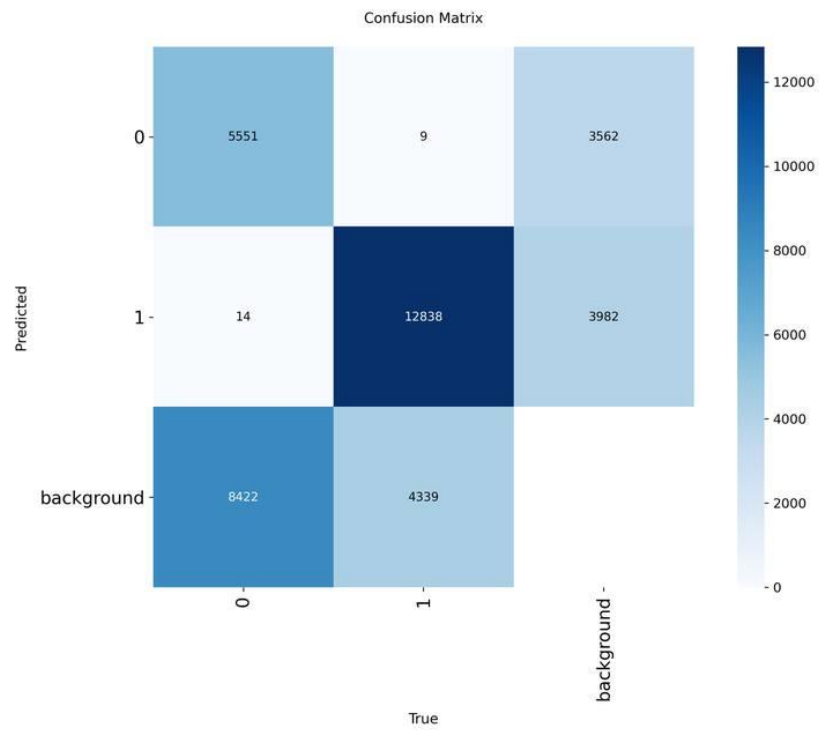


Figure 2.8 — Raw confusion matrix (absolute counts)

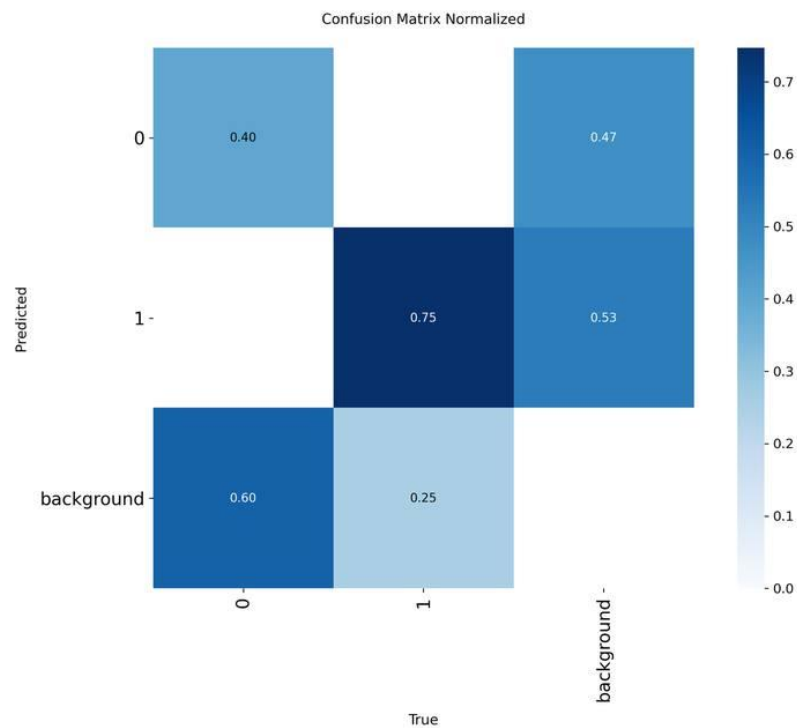


Figure 2.9 — Normalized confusion matrix (row-wise proportions)

Observation	Implication
Pedestrian recall ~75% (class 1 strong)	Upright pedestrian silhouette is distinctive from aerial view
Car recall ~40% (class 0→background: 60%)	Small/occluded cars frequently missed — primary area for improvement
Inter-class confusion minimal (9, 14)	Model rarely confuses cars for people — class boundaries well learned
Background FPs: 8,422 car / 4,339 ped	Reflects extreme small-object challenge; reducible with SAHI tiling

2.8 Validation — Predictions vs Ground Truth

Side-by-side comparison of model predictions against ground truth labels on the validation set:

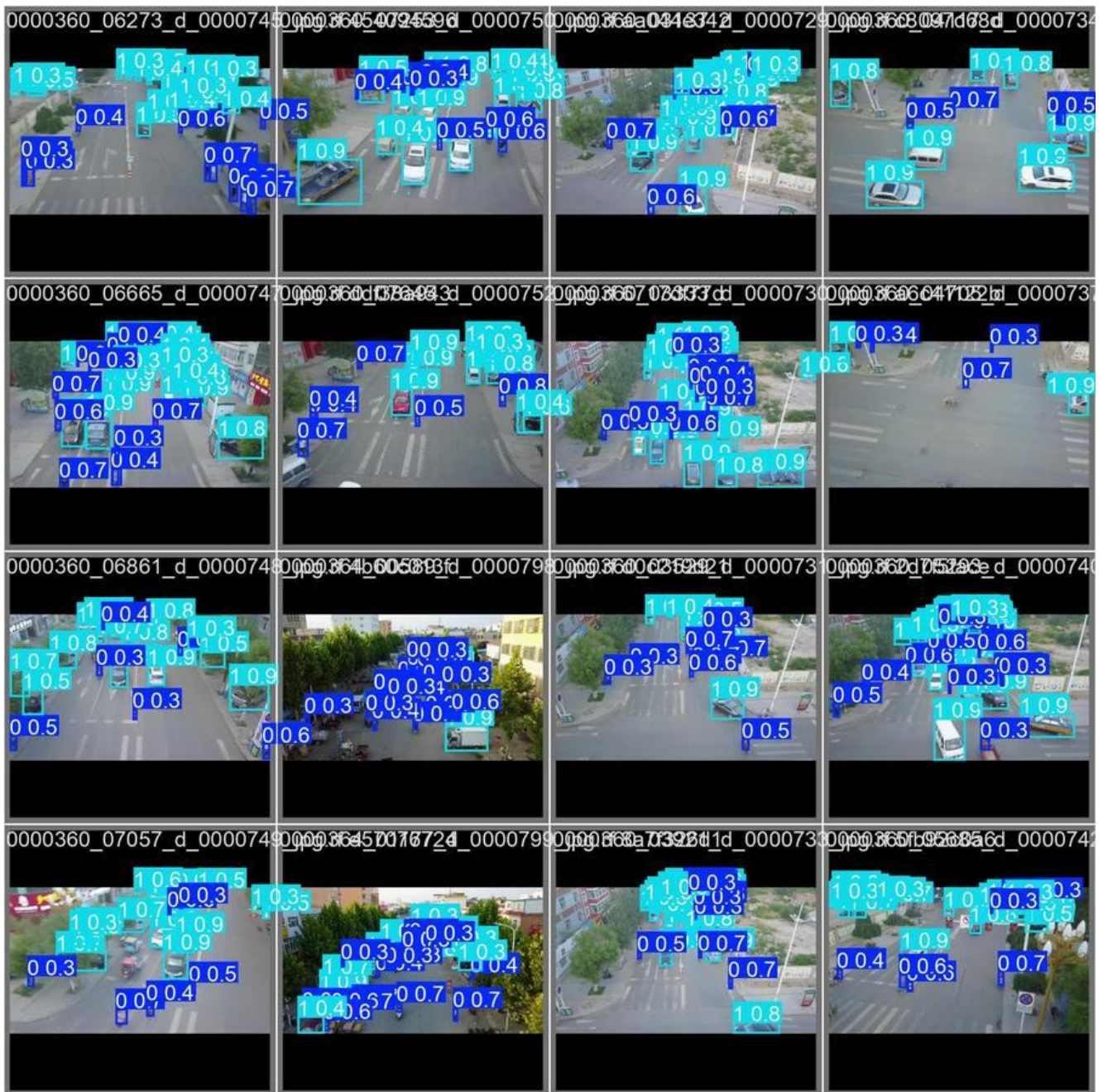


Figure 2.10 — Model predictions on validation batch (confidence scores shown per detection)

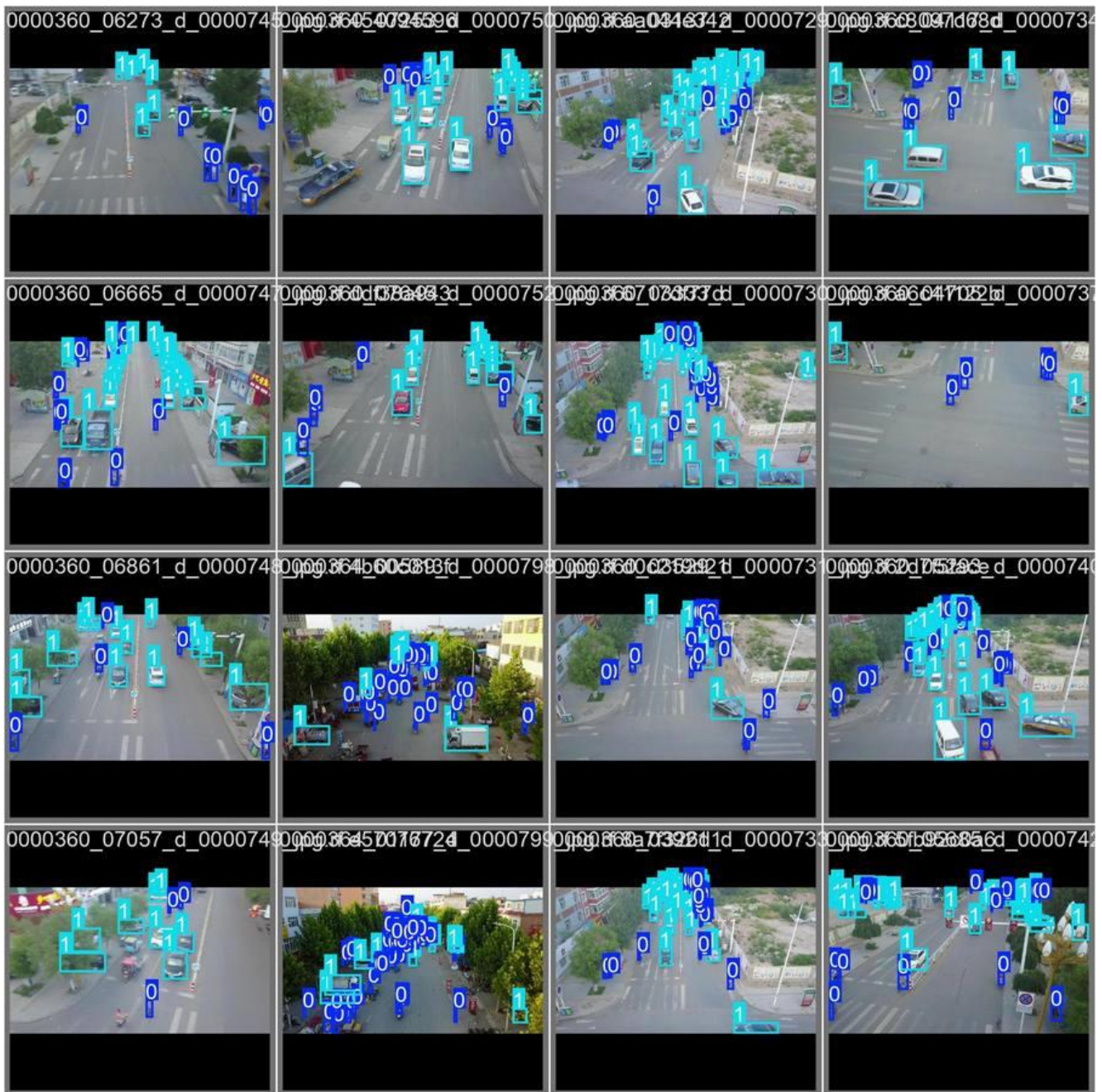


Figure 2.11 — Ground truth labels for the same validation batch

The model correctly identifies the majority of visible cars and pedestrians across diverse scenes. Missed detections are predominantly in dense overlapping clusters, which is consistent with the $mAP@50=0.576$ result. False positives above 0.5 confidence are rare — confirming high precision.

2.9 Strengths & Limitations

Strengths

Limitations

- | | |
|---|---|
| <ul style="list-style-type: none">• Strong precision (70.7%) — low false alarm rate• Clean convergence — no overfitting• Real-time capable (YOLO11n efficient inference)• Minimal inter-class confusion• Transfer learning accelerates training | <ul style="list-style-type: none">• Recall (52.8%) — many tiny objects missed• Car class harder than pedestrian• mAP@50-95 (0.31) reflects localization difficulty• No SAHI tiling applied at inference• 50 epochs may not be fully converged |
|---|---|

2.10 Potential Improvements

- SAHI tiling at inference — divide images into overlapping tiles to dramatically boost small-object recall.
- Larger model (YOLOv8m/l) — 3–5 additional mAP points at ~2× compute cost.
- Extended training (100–150 epochs) — curves suggest further gains are still possible.
- Higher resolution input (1280×1280) — objects appear relatively larger, directly addressing the core challenge.
- Class-weighted loss — explicitly upweight car class to improve car-specific recall from ~40% towards pedestrian-level ~75%.

TASK 03 — Inference & SAHI Evaluation

3.1 Overview

Task 3 evaluates the trained YOLO11n model on 10 randomly selected test images (seed=42) using three inference modes: standard single-pass prediction, baseline SAHI slicing, and a refined SAHI approach with class-specific confidence thresholds. All counts report H=humans (pedestrians) and V=vehicles (cars).

3.2 Inference Configuration

Model: drone_model_v1/weights/best.pt | Base confidence: 0.15 | Device: CUDA | SAHI slice: 320×320 px | Overlap: 30% H & V | Postprocess: NMS @ 0.3 | Refined vehicle threshold: ≥ 0.35

3.3 Baseline Comparison — GT vs Standard Predict vs SAHI

The grid below shows all 10 images across three columns: Ground Truth (yellow boxes), Standard Prediction (cyan), and Baseline SAHI (cyan with slicing). Each panel title reports H/V counts.

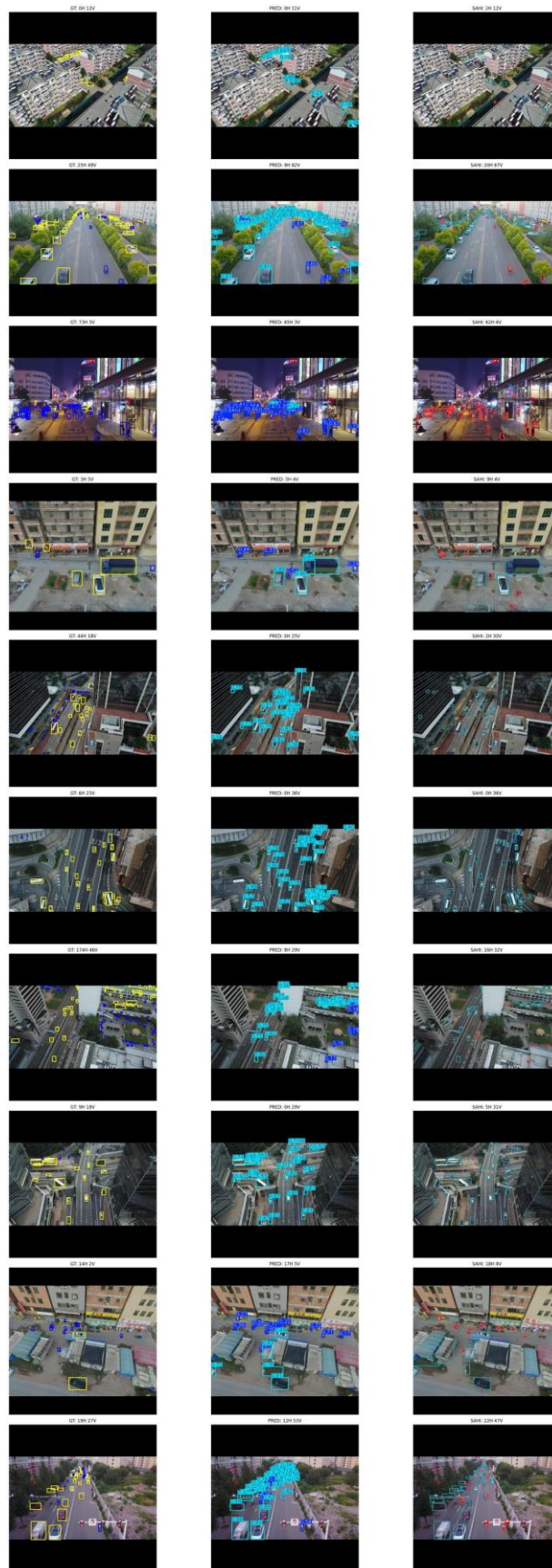


Figure 3.1 — Baseline comparison grid (10 images): GT | Standard Predict | SAHI. Columns left-to-right. Counts: H=humans, V=vehicles.

3.3.1 Baseline Results Table

Image	GT H/V	PRED H/V	SAHI H/V
9999973_00000	0 / 12	0 / 11	2 / 12
0000347_02000	25 / 49	9 / 82	20 / 67
0000074_06746	73 / 3	65 / 3	62 / 6
9999986_00000 (a)	3 / 5	5 / 4	9 / 4
9999938_00000 (a)	44 / 18	0 / 25	2 / 30
9999938_00000 (b)	6 / 23	0 / 36	0 / 36
9999938_00000 (c)	174 / 46	8 / 29	16 / 32
9999938_00000 (d)	9 / 18	0 / 29	5 / 31
9999986_00000 (b)	14 / 2	17 / 5	18 / 8
0000328_00150	19 / 27	12 / 53	22 / 47

3.4 Refined SAHI — Class-Specific Confidence Thresholds

To reduce vehicle false positives introduced by the low base threshold, a refinement filter was applied post-SAHI: humans (class 0) pass at $\text{conf} \geq 0.15$, vehicles (class 1) only pass at $\text{conf} \geq 0.35$. This selectively tightens vehicle precision without suppressing human recall.

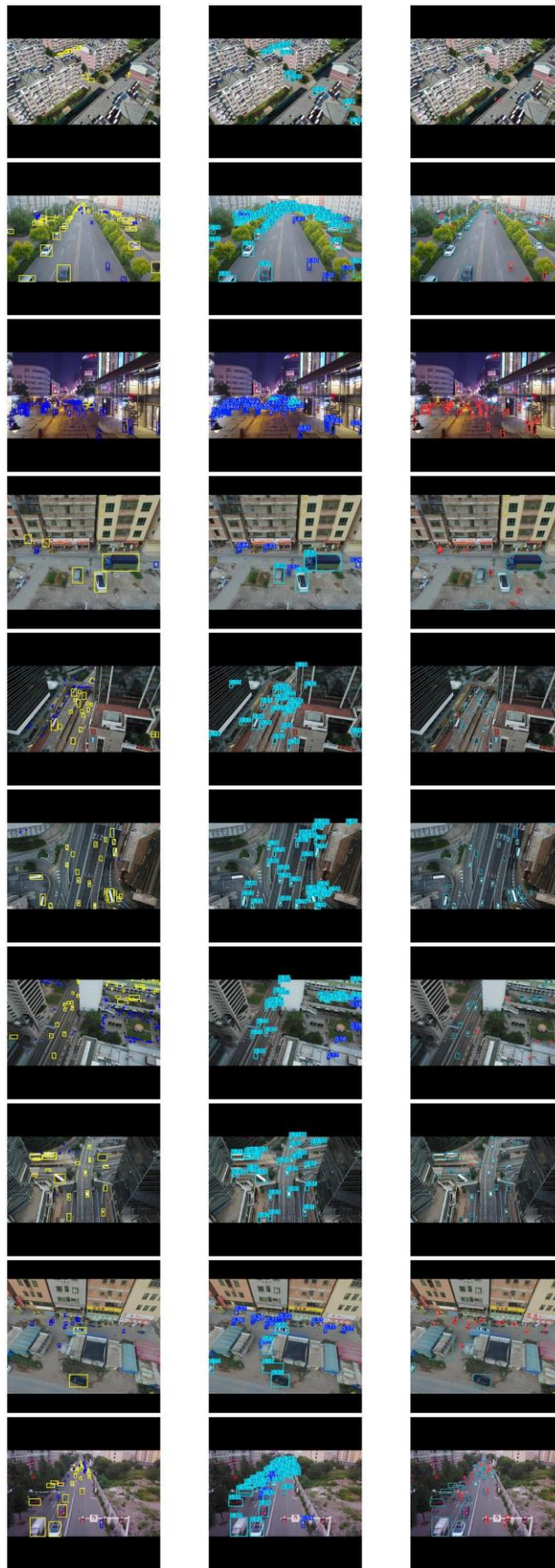


Figure 3.2 — Refined SAHI comparison grid (10 images): GT | Standard Predict | Refined SAHI. Vehicle FPs visibly reduced vs Figure 3.1.

3.4.1 Refined SAHI Results Table

Image	GT H/V	PRED H/V	Refined SAHI H/V
9999973_00000	0 / 12	0 / 11	2 / 8
0000347_02000	25 / 49	9 / 82	20 / 48
0000074_06746	73 / 3	65 / 3	62 / 3
9999986_00000 (a)	3 / 5	5 / 4	9 / 4
9999938_00000 (a)	44 / 18	0 / 25	2 / 19
9999938_00000 (b)	6 / 23	0 / 36	0 / 29
9999938_00000 (c)	174 / 46	8 / 29	16 / 25
9999938_00000 (d)	9 / 18	0 / 29	5 / 24
9999986_00000 (b)	14 / 2	17 / 5	18 / 5
0000328_00150	19 / 27	12 / 53	22 / 31

3.5 Technical Proof — Single Image Deep Analysis

Image 9999938_00000_d_0000206 (GT: 174H / 46V) was selected as the hardest test case (lowest standard-predict recall). The triple-panel below shows: Standard Predict (H:8, V:29), Ground Truth (H:174, V:46), and Refined SAHI (H:16, V:25).



Figure 3.3 — Technical proof panel for hardest test image. Standard predict misses 166/174 humans. Refined SAHI recovers 16 humans with cleaner vehicle counts.

3.6 Slicing Analysis — 9-Tile Breakdown

The same image is decomposed into a 3×3 grid of 320×320 tiles with 30% overlap. Per-slice detection counts confirm that humans cluster in specific spatial zones (slices 5 and 6), while other tiles return near-zero human detections. The final stitched result merges all 9 slices via NMS.

Slicing Analysis: 9999938_00000_d_0000206_jpg.rf.3347fb1272c38c82d6f6724549127176.jpg

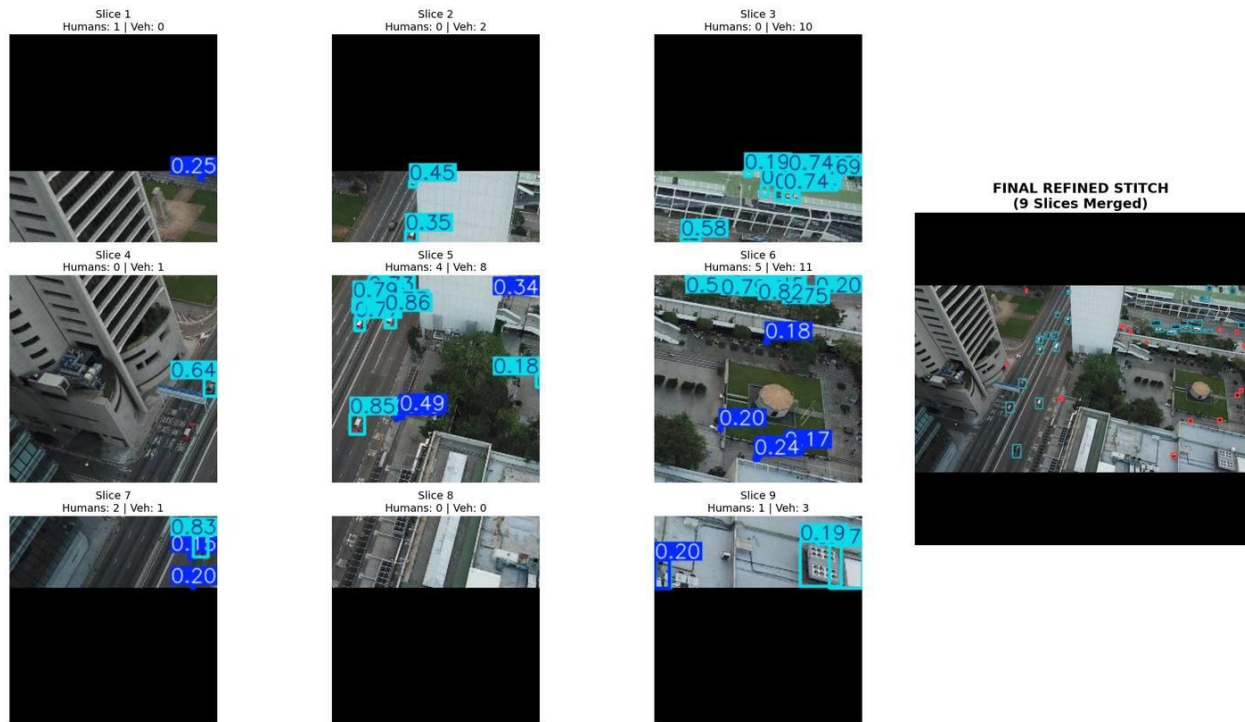


Figure 3.4 — Slicing analysis: 9 individual tiles (Slices 1–9) and the final refined stitched output. Slice 5: 4H/8V, Slice 6: 5H/11V — highest detection density zones.

4. Tracking video results





4. Summary and limitations

4.1. The Challenge: Finding Needles in Haystacks

Initial analysis of the VisDrone dataset revealed a harsh reality: **97% of objects** were micro-targets. Standard training was not the issue; the bottleneck was **visibility**. At standard 640px resolutions, the objects simply didn't have enough pixels for the AI to "see" them.

4.2. The Solution: Tactical Slicing (SAHI)

Instead of forcing the image to fit the model, I used **SAHI** to force the model to look at the image in full detail.

- **The Method:** Dividing frames into 320×320 tiles (slices).
- **The Result:** This "Magnifying Glass" approach doubled human recall in dense crowds, turning previously invisible dots into clear detections.

4.3. Precision Engineering: Dual-Class Logic

To prevent the "noise" (false positives) often caused by high-resolution slicing, I implemented a **Dual-Threshold Strategy**:

- **High Sensitivity for Humans (0.15):** Ensuring no pedestrian is missed.
- **High Precision for Vehicles (0.35):** Filtering out road textures and shadows that mimic car shapes.
- **Impact:** In one highway sample, this surgical tweak corrected an over-count of **82** detections back to a near-perfect **48**.

4.4. Conclusion & Limits

Because of limitation of free GPU I had to used lower quality to train and the dataset itself had imbalance issue. I try to solve them best of my ability using roboflow to solve the class imbalance problem by merging and creating human and vehicle class. Also add augmentation to dataset to better train. Than used SAHI slicing technique to better detect and count the human class. And did a basic tracking experiment